

AdaGrad (Adaptive Gradient) Explained

Optimizers in Deep Learning — Part 4 | Notes based on the CampusX video

1. Introduction: What is AdaGrad?

AdaGrad stands for **Adaptive Gradient**. Its core idea is simple: instead of keeping the learning rate fixed for every parameter throughout training, AdaGrad **adapts (updates) the learning rate** for each parameter individually, based on the situation encountered so far during training.

AdaGrad tends to perform especially well in two common situations:

- **Differently-scaled input features:** For example, if one feature is CGPA (ranging roughly 0–10) and another is package/salary (ranging from 0 to a few lakhs), the very different scales can cause problems for standard optimizers. AdaGrad handles such data well — although in practice, many people simply normalize their features beforehand to avoid this issue in the first place.
- **Sparse features:** This is the more genuinely useful case. A sparse feature is one where most values are zero, with only a few non-zero entries. For example, in a dataset predicting salary package from CGPA and "is this student from IIT," the IIT column would mostly contain zeros (since relatively few students are from IIT) with occasional ones. When such sparse columns are present, AdaGrad gives noticeably better results than other optimizers.

The natural question is: what actually goes wrong when features are sparse? That problem is called the **Elongated Bowl Problem**, explained next.

2. The Elongated Bowl Problem

Consider plotting the loss surface with respect to two parameters, w and b . If the input feature (corresponding to w) takes a wide range of "normal" values, the resulting loss contour plot tends to look roughly **circular**. But if that input feature is sparse — mostly zeros, with very few non-zero values — the contour plot becomes **elongated**, stretched out along one axis like a long, narrow valley.

In this elongated bowl, the loss changes sharply along one axis but stays nearly flat (constant) along the other axis over long stretches. This creates a **valley**-shaped loss surface: change happens in one direction, while the other direction sees almost no meaningful change.

Standard optimization algorithms — vanilla gradient descent, Momentum, or NAG — struggle to navigate this kind of valley efficiently. This is precisely the problem that AdaGrad is designed to solve.

3. How Existing Optimizers Behave on Sparse Data

Using the same sparse dataset, the video visualizes how Batch Gradient Descent and Momentum actually move through this elongated valley.

- **Batch Gradient Descent:** The optimizer first moves almost entirely along the b -axis, descending into the valley, and only afterward starts moving along the w -axis toward the minimum. This sequential, bent path wastes time — a direct diagonal path would have been much faster.
- **Momentum:** Momentum moves quickly at first and overshoots in the b -direction, then has to curve back toward the minimum. It clearly is not taking the most direct route either, even though logically a faster, more direct path is visibly possible.

Why does movement happen mostly along the b -axis early on? Because b 's gradient corresponds to a term that is consistently large (effectively multiplied by 1 each time), while w 's gradient is consistently multiplied by very small, mostly-zero values (since the corresponding feature is sparse) — so early on, essentially no change happens along the w -axis, and almost all motion happens along b .

In short: with this kind of data, both Batch Gradient Descent and Momentum perform poorly. Early on, nearly all movement happens in one direction (corresponding to the dense/normal feature) and almost none happens in the other direction (corresponding to the sparse feature) — only later does movement shift to the second direction.

4. Why Sparse Columns Cause Slow Updates

To understand why, consider a single-neuron network with weight w , bias b , and inputs x , producing prediction y_{pred} . Using Batch Gradient Descent with a mean-squared-error loss (as in linear regression), the gradients work out to:

$$\partial L / \partial w \propto (y_{\text{pred}} - y) \cdot x \quad | \quad \partial L / \partial b \propto (y_{\text{pred}} - y) \cdot 1$$

Notice the key difference: the gradient with respect to w is multiplied by the input value x , while the gradient with respect to b is effectively multiplied by 1 (since the bias term's corresponding "input" is always 1).

Now, when you sum these gradients across all training rows (say, 100 rows) to compute the batch update: for b , you're always adding numbers multiplied by 1, so the sum stays a normal-sized number. But for w , if the feature x is sparse (mostly zero), most of the 100 terms being summed are themselves zero — so the total sum ends up being a **very small number**.

A small gradient sum means a small weight update. So the parameter tied to the sparse feature barely moves, while the parameter tied to the dense feature (or the bias) moves normally. This imbalance is exactly why the optimizer's path looks bent: it moves a lot in one direction and barely at all in the other, until the sparse-direction gradient eventually accumulates enough to start mattering.

5. The AdaGrad Solution: Per-Parameter Learning Rates

In every optimization algorithm covered so far (Batch Gradient Descent, Momentum, NAG), all parameters share the **same learning rate**. AdaGrad's central insight: instead of using one global learning rate, use a **different learning rate for each parameter**.

The rule for adjusting each parameter's learning rate is intuitive: if a parameter's gradient has historically been **large**, shrink its learning rate (so its updates stay small and comparable to others). If a

parameter's gradient has historically been **small**, keep its learning rate relatively larger (so its updates aren't left behind). The goal is to make updates across all parameters more comparable in scale, rather than letting dense features dominate and sparse features lag far behind.

6. Mathematical Formulation

The AdaGrad weight update rule is:

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \left[\alpha / \sqrt{(\mathbf{v}_t + \epsilon)} \right] \cdot \nabla L(\mathbf{w}_t)$$

where the accumulator term is:

$$\mathbf{v}_t = \mathbf{v}_{t-1} + \left[\nabla L(\mathbf{w}_t) \right]^2$$

Symbol	Meaning
$\mathbf{w}_t$	The parameter being updated, at step t
α (alpha)	The base (global) learning rate
$\mathbf{v}_t$	Running sum of squared gradients for this parameter, accumulated across all steps so far
ϵ (epsilon)	A very small constant added purely to avoid division by zero when $\mathbf{v}_t = 0$
$\nabla L(\mathbf{w}_t)$	The gradient of the loss with respect to this parameter, at step t

Notice that this formula is really just the standard gradient descent update, with one change: the learning rate α is divided by $\sqrt{(\mathbf{v}_t + \epsilon)}$. The gradient is squared (rather than used directly) specifically so that its sign doesn't matter — only its magnitude accumulates, since AdaGrad cares about how large the historical gradient has been, not which direction it pointed.

This produces exactly the adaptive behavior described above: if a parameter's gradients have been large historically, $\mathbf{v}_t$ grows large, so dividing α by a large number shrinks the effective learning rate. If a parameter's gradients have been small (as with a sparse feature), $\mathbf{v}_t$ stays small, so the effective learning rate stays relatively larger — helping that parameter keep pace.

7. Effect of AdaGrad on Sparse Data

When AdaGrad is applied to the same sparse dataset and compared side by side with a standard optimizer, the improvement is clear: AdaGrad takes a noticeably more direct path toward the global minimum, rather than the bent, inefficient path seen earlier with plain Gradient Descent or Momentum.

8. Disadvantage of AdaGrad

AdaGrad does have a significant disadvantage — significant enough that AdaGrad is generally **not used for training neural networks** in practice. It works reasonably well for simpler problems like linear regression, but not for complex neural networks.

The issue: AdaGrad's accumulator v_t is a running **sum** of squared gradients, so it only ever grows larger over the course of training, no matter how many epochs you run. As v_t keeps growing, the effective learning rate (α divided by a growing number) keeps shrinking. Eventually the learning rate becomes so tiny that updates become negligible, and the optimizer effectively stops moving — getting stuck before it actually reaches the minimum.

This is the biggest problem with AdaGrad: it often fails to converge all the way to the global minimum, because its learning rate decays too aggressively and too permanently over time.

9. Summary

AdaGrad adapts the learning rate separately for each parameter, based on the historical magnitude of that parameter's gradients. Parameters with large historical gradients get a smaller effective learning rate, while parameters with small historical gradients (such as those tied to sparse features) retain a relatively larger one. This makes AdaGrad especially effective for data with sparse features, where it solves the "elongated bowl" problem that causes standard optimizers to take slow, bent paths to the minimum. However, because its gradient accumulator only ever grows, the effective learning rate keeps shrinking indefinitely, which can cause training to stall before reaching the true minimum. This is why AdaGrad is rarely used for deep neural networks in practice — a limitation addressed by later optimizers such as RMSProp and Adam.

Reference: Video Timestamps

Source video: "AdaGrad Explained in Detail with Animations | Optimizers in Deep Learning Part 4" (CampusX)

- 00:00 — Intro
- 00:15 — Adaptive Gradient Introduction
- 03:42 — Elongated Bowl Problem
- 07:22 — Visual Representation
- 09:42 — How do Optimizers Behave?
- 17:22 — Mathematical Intuition
- 24:20 — Disadvantage
- 26:16 — Outro

Notes compiled for personal study based on the CampusX YouTube lecture on AdaGrad (Adaptive Gradient). For the full explanation with animations, refer to the original video.